TITLE: Parallel Matrix Multiplication Using MPI + CUDA on Multiple GPUs

Problem Statement: Write a MPI + CUDA program for Parallel Matrix Multiplication using multiple GPUs.

Aim: The main aim of this program is to understand how parallel computing works using MPI and CUDA for Matrix Multiplication.

This assignment helps to understand:

- How matrix multiplication is performed.
- How tasks are divided in parallel computing.
- How MPI creates multiple parallel processes.
- How CUDA uses GPU threads for parallel execution.
- How CPU and GPU work together.
- How parallel processing improves execution speed.
- Why GPUs are faster for large mathematical computations.

Technology Used:

- MPI (Message Passing Interface)
- CUDA C/C++
- NVIDIA GPU
- CUDA Toolkit
- OpenMPI

Platforms:

- Ubuntu Linux
- Google Colab
- NVIDIA GPU Systems

Steps to install Tools:

FOR UBUNTU LINUX

Step 1 : Update Ubuntu Packages

- sudo apt update
- sudo apt upgrade -y

Step 2 : Install Build Essentials

- sudo apt install build-essential -y

Step 3 : Install NVIDIA CUDA Toolkit

- sudo apt install nvidia-cuda-toolkit -y

Step 4 : Verify CUDA Installation

- nvcc --version

Step 5 : Install MPI Library (OpenMPI)

- sudo apt install openmpi-bin openmpi-common libopenmpi-dev -y

Step 6 : Verify MPI Installation

- mpirun --version

Step 7 : Check Available GPUs

- nvidia-smi

Step 8 : Create CUDA File

- nano matrix_mul_mpi_cuda.cu

Write the code and save the file

Step 9 : Compile the Program

- mpicxx matrix_mul_mpi_cuda.cu -o matrix_mul -lcudart

Step 10 : Run the Program

- mpirun -np 2 ./matrix_mul

FOR GOOGLE COLAB

Step 1 : Enable GPU

Runtime → Change Runtime Type → Hardware Accelerator → GPU

Step 2 : Check GPU

- !nvidia-smi

Step 3 : Verify CUDA

- !nvcc --version

Step 4 : Install MPI

- !apt-get install openmpi-bin openmpi-common libopenmpi-dev -y

Step 5 : Create CUDA File

- %%writefile matrix_mul_mpi_cuda.cu

Write the code below this command.

Step 6 : Compile the Program

- !mpicxx matrix_mul_mpi_cuda.cu -o matrix_mul -lcudart

Step 7 : Run the Program

- !mpirun -np 2 ./matrix_mul

INPUT :

Matrix A -> [[1, 2],
[3, 4]]

Matrix B -> [[5, 6],
[7, 8]]

CODE :

```
#include <stdio.h>
#include <cuda.h>
#include <mpi.h>

#define N 2

__global__ void matrixMul(int *A, int *B, int *C)
{
   int row = threadIdx.y;
   int col = threadIdx.x;

   int sum = 0;

   for(int k = 0; k < N; k++)
   {
      sum += A[row * N + k] * B[k * N + col];
   }

   C[row * N + col] = sum;
}

int main(int argc, char *argv[])
{
   int rank, size;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   int h_A[N*N] = {1,2,3,4};
   int h_B[N*N] = {5,6,7,8};
   int h_C[N*N];
```

```
    int *d_A, *d_B, *d_C;

    cudaSetDevice(rank);

    cudaMalloc((void**)&d_A, N*N*sizeof(int));
    cudaMalloc((void**)&d_B, N*N*sizeof(int));
    cudaMalloc((void**)&d_C, N*N*sizeof(int));

    cudaMemcpy(d_A, h_A, N*N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, N*N*sizeof(int), cudaMemcpyHostToDevice);

    dim3 threads(N, N);

    matrixMul<<<1, threads>>>(d_A, d_B, d_C);

    cudaMemcpy(h_C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);

    if(rank == 0)
    {
        printf("Result Matrix:\n");

        for(int i = 0; i < N; i++)
        {
            for(int j = 0; j < N; j++)
            {
                printf("%d ", h_C[i*N + j]);
            }
            printf("\n");
        }
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    MPI_Finalize();

    return 0;
}
```

NOTE: For the better visualization use large values of N

OUTPUT :

```
:sanvi~$ nvcc -ccbin mpicxx matrix_mul_mpi_cuda.cu -o matrix_mul
:sanvi~$ mpirun -np 2 ./matrix_mul
Result Matrix:
19 22
43 50
```

EXPLANATION :

This program performs Matrix Multiplication using MPI and CUDA on multiple GPUs.

Matrix Multiplication Formula :

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

Where
A = First Matrix
B = Second Matrix
C = Result Matrix

Step-by-Step Calculation :

C[0][0] = (1×5) + (2×7)
= 5 + 14
= 19

C[0][1] = (1×6) + (2×8)
= 6 + 16
= 22

C[1][0] = (3×5) + (4×7)
= 15 + 28
= 43

C[1][1] = (3×6) + (4×8)
= 18 + 32
= 50

Significance of MPI:

MPI is used for process-level parallelism.

In this program:

- Each MPI process can use one GPU.
- Work can be divided among multiple GPUs.
- Multiple processes execute computations in parallel.

MPI helps in:

- Parallel execution of tasks
- Better workload distribution
- Efficient utilization of multiple GPUs
- Faster execution for large matrix computations

Without MPI:

- Only one process executes the entire computation.

With MPI:

- Multiple processes execute computations simultaneously.

Thus MPI improves scalability and performance.

Significance of CUDA :

CUDA is NVIDIA's GPU programming platform used for GPU parallel computing.

In this program:

- CUDA performs matrix multiplication on GPU.
- Thousands of GPU threads execute computations simultaneously.
- Each CUDA thread computes one matrix element.

Thread	**Computation**
Thread (0,0)	Computes C[0][0]
Thread (0,1)	Computes C[0][1]
Thread (1,0)	Computes C[1][0]
Thread (1,1)	Computes C[1][1]

HOW TASK DIVISION HAPPENS :

Step	**Function**	**Description**
1.	MPI_Init(&argc, &argv);	MPI creates multiple parallel processes.
2.	cudaSetDevice(rank);	Each MPI process gets a GPU according to its rank.

3.	matrixMul<<<1, threads>>>(d_A, d_B, d_C);	CUDA creates multiple GPU threads.

Step 4 : Threads Compute Matrix Elements

Thread	**Output Element**
Thread (0,0)	C[0][0]
Thread (0,1)	C[0][1]
Thread (1,0)	C[1][0]
Thread (1,1)	C[1][1]

Thus:

- MPI divides tasks among processes and GPUs.
- CUDA divides tasks among GPU threads.

MPI WORKING :

Function	**Description**
MPI_Init(&argc, &argv);	Initializes MPI environment..
MPI_Comm_rank(MPI_COMM_WORLD, &rank);	Each process gets a unique rank number.
MPI_Comm_size(MPI_COMM_WORLD, &size);	Gets total number of MPI processes.
cudaSetDevice(rank);	Allocates memory on GPU.

CUDA WORKING:

Function	**Description**
cudaMalloc()	Allocates memory on GPU.
cudaMemcpy()	Copies matrices from CPU memory to GPU memory.
matrixMul<<<1, threads>>>(d_A, d_B,	GPU threads execute matrix

d_C);	multiplication in parallel.
cudaMemcpy()	Copies result matrix back from GPU to CPU memory.

Kernel Launch:
matrixMul<<<1, threads>>>(d_A, d_B, d_C);

Matrix Size	**Kernel Configuration**	**Reason**
2×2	<<<1,(2, 2)>>>	4 threads handle 4 elements
32×32	<<<1,(32,32)>>>	One thread computes one element
1024×1024	Multiple blocks required	Large matrices need more threads

OBSERVATION :

- Matrix multiplication runs in parallel.
- MPI distributes work among GPUs.
- CUDA performs fast GPU computation.
- Multiple GPUs improve performance for large matrices.

CONCLUSION :

This program demonstrates Parallel Matrix Multiplication using MPI and CUDA on multiple GPUs. MPI handles process-level parallelism, while CUDA performs GPU-based parallel computation. Each MPI process uses a separate GPU, resulting in faster execution and efficient utilization of GPU resources.